

Nagy házifeladat: Italrecept



Figure 1: Bár ahol az ital receptek készülnek

Tartalomjegyzék

Maga a programról.....	4
Programozói dokumentáció.....	4
Gyakorlati kivitelezés.....	5
Recipe osztály, RecipeIngredient osztály.....	6
RecipeDatabase.....	7
Falj kezelés.....	7
Recipeingridient osztály.....	8
getAmount().....	8
getIngredient().....	8
getName().....	8
toString().....	9
Recipedatabase osztály.....	9
findIngredient().....	9
addIngredient().....	9
addRecipe().....	9
deleteRecipe().....	9
findRecipe().....	10
listIngredientsOfRecipe().....	10
listRecipesByName().....	10
listRecipesByIngredientCount().....	10
saveToFile().....	10
loadFromFile().....	10
Recipe osztálya.....	10
getName().....	11
getIngredients().....	11
addIngredient().....	11
removeIngredient().....	11
findIngredient().....	11
listIngredients().....	11
toString().....	11
Menu osztály.....	12
isNumber(String s).....	12
menu_print().....	12
userInput().....	12
InputIngridient().....	12
InputRecipe().....	12
newRecipe().....	13
newinput().....	13
Ingridient osztálya.....	13
getName().....	13
getUnit().....	13
toString().....	13
Appservice osztály.....	14
Newingredient().....	14
NewRecipe().....	14
DeleteRecipe().....	14

ListIngredients().....	14
SaveToFile().....	14
LoadFromFile().....	15
Felhasználói kézikönyv – Használati útmutató.....	15
A program indítása.....	15
Főmenü működése.....	15
Új alapanyag hozzáadása.....	16
Új recept hozzáadása.....	16
Recept törlése.....	16
Hozzávalók listázása.....	16
Receptek listázása név szerint.....	16
Receptek listázása hozzávalók száma szerint.....	16
Mentés fájlba.....	17
Betöltés fájlból.....	17
Kilépés.....	17

Maga a programról:

A program célja italreceptek és azok alapanyagainak nyilvántartása egy egyszerű, objektumorientált Java alkalmazás segítségével, amelyben tesztelési funkciók (JUnit) is helyet kapnak. A projekt lehetővé teszi az alapanyagok rendszerezett tárolását, valamint ezekből különböző receptek összeállítását és kezelését, figyelembe véve, hogy egy ital több összetevőből épül fel. A rendszer kisebb mennyiségű adat kezelésére készült, amelyet fájlba mentéssel és visszatöltéssel valósít meg, ugyanakkor a felépítése rugalmas, így könnyen bővíthető további receptekkel és alapanyagokkal.

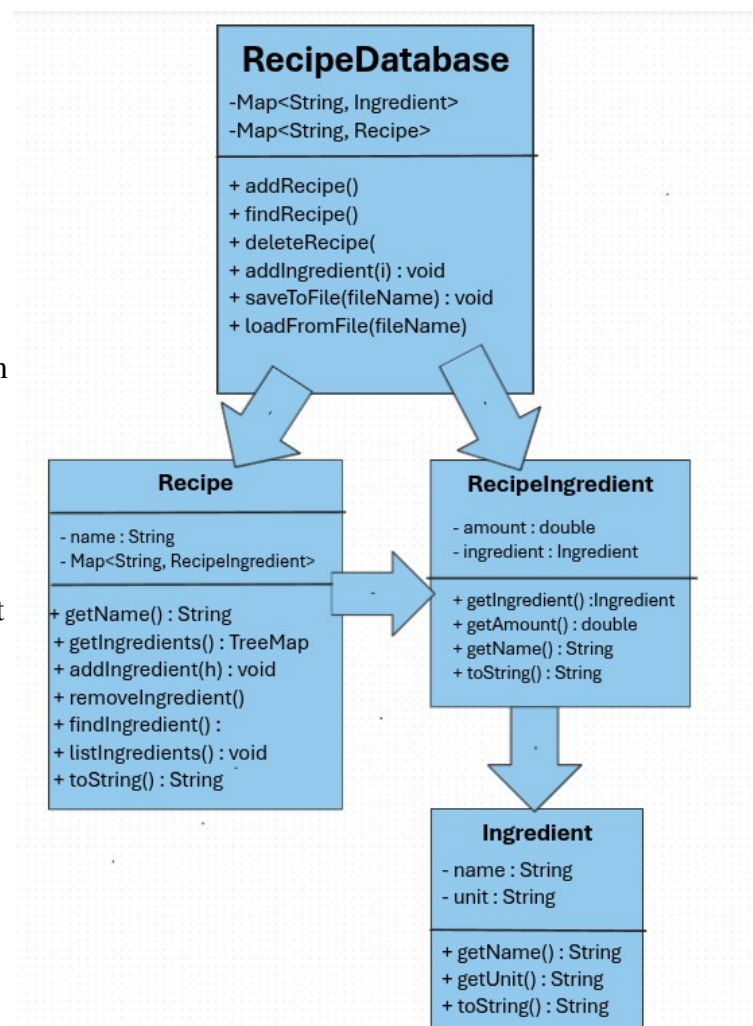
A felhasználó a konzolos felületen keresztül új adatokat vihet fel, megtekintheti egy adott recept hozzávalóit, törölhet már meglévő recepteket, valamint az aktuális adatállapotot elmentheti vagy visszatöltheti fájlból. A program működése során kiemelt szerepet kap az objektumorientált tervezés, a jól strukturált osztályok használata, a gyűjtemények (például térképek) alkalmazása, valamint a fájlkezelés és a tesztelhetőség biztosítása.

Összességében egy olyan gyakorlati alkalmazásról van szó, amely a mindennapi életben is hasznos lehet, hiszen egyfajta digitális receptgyűjteményként szolgál, amely rendszerezetten és könnyen kezelhető módon tárolja az italokhoz tartozó információkat.

Programozói dokumentáció

A Java projekt több különálló osztályból épül fel, amelyek mindegyike külön feladatot lát el a program működésében. Az Ingredient osztály az alapanyagokat reprezentálja, vagyis eltárolja az alapanyag nevét és mértékegységét. A

RecipeIngredient osztály egy receptben szereplő konkrét hozzávalót ír le, amely egy alapanyagból és egy mennyiségi értékből áll. A Recipe osztály magát az italreceptet valósítja meg: tartalmazza a recept nevét, valamint a hozzá tartozó hozzávalókat egy TreeMap adatszerkezetben.



A program központi adattároló osztálya a RecipeDatabase, amely az összes felvitt alapanyagot és receptet kezeli. Ebben található azok a metódusok, amelyekkel új alapanyagot vagy receptet lehet hozzáadni, meglévő receptet lehet törölni, illetve név alapján keresni lehet az adatok között. A fájlba mentés és fájlból betöltés szintén ebben az osztályban kapott helyet, szerializáció segítségével. Ez azt jelenti, hogy a program az aktuális adatbázis-objektumot egyben menti el egy .dat fájlba, majd később ugyanebből vissza tudja állítani az adatokat.

A felhasználóval való kommunikációért a Menu osztály felel. Ebben található az adatbekérő és ellenőrző metódusok, például a menüpont kiválasztása, új alapanyag bekérése, receptnév bekérése, illetve a recept hozzávalóinak megadása. Az AppService osztály a program vezérlési logikáját fogja össze: innen hívódnak meg az olyan műveletek, mint az új alapanyag felvétele, új recept létrehozása, recept törlése, hozzávalók listázása, mentés és betöltés. A Main osztály a program belépési pontja, vagyis innen indul az alkalmazás. Itt jön létre a menü és az adatbázis, majd egy ciklusban a felhasználó választása alapján a megfelelő szolgáltatás fut le.

A projekt működése nagyobb egységekre bontható: adatmodellek kezelése, felhasználói adatbekérés, adatbázis-műveletek, fájlkezelés, valamint tesztelés. Az adatmodelleket az Ingredient, RecipeIngredient és Recipe osztályok adják, az adatbázis-kezelést a RecipeDatabase, a felhasználói kommunikációt a Menu, a programlogikát pedig az AppService és a Main osztályok valósítják meg.

Gyakorlati kivitelezés

A programban a TreeMap használata tudatos tervezési döntés volt, mivel az alkalmazásban az alapanyagokat és recepteket név alapján kell eltárolni, keresni és törölni. A TreeMap kulcs-érték párokban tárolja az adatokat, ahol a kulcs egy String, például az alapanyag vagy recept neve, az érték pedig maga az objektum. Ez jól illeszkedik a program működéséhez, hiszen a felhasználó is név alapján adja meg az alapanyagokat és recepteket.

A TreeMap egyik nagy előnye, hogy az elemeket automatikusan rendezett sorrendben tartja a kulcsok alapján. Ez azt jelenti, hogy az alapanyagok vagy hozzávalók listázásakor az adatok átlátható, ABC sorrendben jelennek meg. Ez külön rendező algoritmus nélkül is rendezett megjelenítést biztosít, ami egyszerűbbé és tisztábbá teszi a kódot.

A programban például a RecipeDatabase osztály két TreeMap adatszerkezetet használ: az egyik az alapanyagokat, a másik a recepteket tárolja. A Recipe osztályban szintén TreeMap található, amely az adott recept hozzávalóit kezeli. Ez azért előnyös, mert egy konkrét alapanyag vagy recept gyorsan megtalálható a nevének megadásával, például findIngredient() vagy findRecipe() metóduson keresztül.

Összességében a TreeMap használata azért megfelelő választás ebben a projektben, mert támogatja a név alapú keresést, törlést és hozzáadást, miközben automatikusan rendezett struktúrát biztosít. A feladat jellegéhez jobban illeszkedik, mint egy egyszerű lista, mert nem index alapján kell kezelni az adatokat, hanem név szerint, ami természetesebb egy receptkezelő alkalmazás esetében.

A program heterogén adatszerkezetet is alkalmaz, mivel nem egyszerű szövegeket vagy számokat tárol, hanem egymásba épülő, különböző típusú objektumokat. Egy RecipeDatabase objektum Recipe

és Ingredient objektumokat tárol, egy Recipe objektum több RecipeIngredient objektumot tartalmaz, a RecipeIngredient pedig egy Ingredient objektumot és egy mennyiséget kapcsol össze. Ez jól szemlélteti az objektumorientált modellezést, hiszen az italreceptek valódi felépítését követi: egy recept több hozzávalóból áll, egy hozzávaló pedig egy alapanyagból és mennyiségből épül fel.

A fájlba írás és fájlból olvasás szerializálással történik. Ennek segítségével a program nem külön soronként menti el az adatokat szöveggént, hanem az egész RecipeDatabase objektumot írja ki fájlba. Így a benne lévő receptek, alapanyagok és azok kapcsolatai egyben megőrizhetők. A mentés az ObjectOutputStream, a betöltés pedig az ObjectInputStream segítségével történik. Emiatt az érintett osztályok Serializable interfészt valósítanak meg, hogy Java objektumként menthetők és visszatölthetők legyenek.

A tesztelés JUnit segítségével valósul meg. A projektben külön tesztosztályok ellenőrzik az egyes osztályok működését, például az alapanyag nevének és mértékegységének lekérdezését, recepthez hozzávaló hozzáadását, hozzávaló törlését, recept keresését és adatbázisba való felvételét. A követelmény szerint legalább három osztály összesen tíz metódusát kell tesztelni, ezért a tesztek többek között az Ingredient, Recipe és RecipeDatabase osztályokra épülnek.

A felhasználói felület egyszerű konzolos menürendszerként működik. A cél nem egy látványos grafikus felület készítése volt, hanem az, hogy a felhasználó könnyen ki tudja választani a szükséges műveletet: új alapanyag felvétele, új recept létrehozása, recept törlése, hozzávalók listázása, mentés, betöltés vagy kilépés. A program így elsősorban az objektumorientált felépítésre, a megfelelő adatszerkezetekre, a fájlkezelésre és a tesztelhetőségre helyezi a hangsúlyt.

A program fő logikája egy objektumorientált italrecept-kezelő rendszer köré épül, amelyben az adatok nem egyszerű szöveggént vannak eltárolva, hanem egymással kapcsolatban álló objektumokként. A rendszer alapegysége az Ingredient osztály, amely egy alapanyagot ír le. Egy alapanyaghoz tartozik egy név és egy mértékegység, például rum és l, vagy cukor és g. Ez az osztály önmagában még csak egy egyszerű adatmodell, amely eltárolja az alapanyag legfontosabb tulajdonságait, majd getter metódusokon keresztül visszaadja ezeket az értékeket. Az osztály Serializable, vagyis az objektumai fájlba menthetők és később visszatölthetők.

Recipe osztály, RecipeIngredient osztály

A receptekhez azonban nem elég csak az alapanyag neve és mértékegysége, mert egy receptben az is fontos, hogy az adott alapanyagból mennyi szükséges. Ezt a szerepet tölti be a RecipeIngredient osztály. Ez összekapcsol egy Ingredient objektumot egy mennyiséggel, tehát például azt tudja eltárolni, hogy egy recepthez 2 l rum vagy 5 db alma szükséges. A konstruktorban ellenőrzés is található: ha az alapanyag null, akkor a program hibát dob, mert nem lehet olyan recept-hozzávalót létrehozni, amely mögött nincs valódi alapanyag. Ez azért fontos, mert így a program belső állapota következetes marad. A RecipeIngredient tehát egy köztes objektum: nem maga az alapanyag, és nem is maga a recept, hanem a kettő közötti kapcsolat mennyiséggel kiegészítve.

A Recipe osztály képviseli magát az italreceptet. Egy receptnek van neve, például Mojito, és tartalmaz több hozzávalót. Ezeket a hozzávalókat a program egy TreeMap<String, RecipeIngredient> adatszerkezetben tárolja. A TreeMap kulcsa a hozzávaló neve, az értéke pedig a konkrét

RecipeIngredient objektum. Ez a megoldás azért előnyös, mert egy hozzávaló név alapján gyorsan megkereshető, törölhető vagy felülírható. Emellett a TreeMap automatikusan rendezi a kulcsokat, ezért a hozzávalók listázásakor az elemek rendezett sorrendben jelenhetnek meg. A Recipe osztály metódusai ennek megfelelően a recept belső kezelését végzik: hozzá lehet adni új hozzávalót, törölni lehet meglévőt, keresni lehet egy adott hozzávalót, illetve ki lehet listázni a recept teljes tartalmát.

RecipeDatabase

A program központi eleme a RecipeDatabase osztály, amely az egész alkalmazás adatbázisaként működik. Ez az osztály két külön TreeMap-et tartalmaz: az egyik az alapanyagokat tárolja, a másik a recepteket. Az alapanyagoknál a kulcs az alapanyag neve, az érték pedig az Ingredient objektum. A recepteknél ugyanígy a recept neve a kulcs, az érték pedig maga a Recipe objektum. Ennek köszönhetően a programban minden fontos adat név alapján kezelhető. Ha a felhasználó megadja egy alapanyag vagy recept nevét, akkor az adatbázis közvetlenül meg tudja keresni a megfelelő objektumot. Ez sokkal természetesebb megoldás egy receptkezelő programnál, mint az index alapú keresés, mert a felhasználó sem sorszámokban, hanem nevekben gondolkodik.

A recept létrehozásának logikája úgy épül fel, hogy először az alapanyagokat kell felvinni az adatbázisba. Amikor később a felhasználó egy új receptet készít, a program nem új, ismeretlen alapanyagokat pakol bele közvetlenül a receptbe, hanem először megkeresi, hogy az adott alapanyag szerepel-e már az adatbázisban. Ha igen, akkor az ebből létrehozott Ingredient objektum bekerül egy RecipeIngredient objektumba a megadott mennyiséggel együtt, majd ez hozzáadódik a recepthez. Ha az alapanyag nem található, akkor a program jelzi, hogy előbb fel kell venni az adott alapanyagot. Ez a megoldás biztosítja, hogy a receptek csak létező, korábban rögzített alapanyagokra épüljenek, vagyis az adatmodell konzisztens marad.

A rendszerben a heterogén adatszerkezet gyakorlati használata is jól megjelenik. Nem egyetlen listában tárolódnak egyszerű szövegek, hanem különböző típusú objektumok kapcsolódnak egymáshoz. A RecipeDatabase Ingredient és Recipe objektumokat tárol, a Recipe RecipeIngredient objektumokat tartalmaz, a RecipeIngredient pedig egy Ingredient objektumra hivatkozik és mellette egy mennyiséget tárol. Ez a felépítés közelebb áll a valós világhoz: egy recept több hozzávalóból áll, egy hozzávaló pedig egy konkrét alapanyagból és egy mennyiségi értékből. Így a program nemcsak adatokat tárol, hanem a köztük lévő kapcsolatokat is modellezi.

Falj kezelés

A fájlkezelés a RecipeDatabase osztályban valósul meg serializáció segítségével. A saveToFile() metódus az aktuális adatbázis-objektumot egyben írja ki fájlba ObjectOutputStream használatával. Ez azt jelenti, hogy nem külön soronként kerülnek elmentésre az alapanyagok és receptek, hanem az egész objektumstruktúra kerül fájlba. A loadFromFile() ennek az ellentéte: ObjectInputStream segítségével beolvassa a korábban elmentett objektumot, majd visszaalakítja RecipeDatabase típusúvá. Emiatt minden érintett osztály Serializable, hiszen csak így lehet őket Java objektumként fájlba menteni és később visszatölteni.

Összességében a program fő működése úgy írható le, hogy a felhasználó először alapanyagokat hoz létre, ezek bekerülnek a központi adatbázisba, majd ezekből az alapanyagokból recepteket állít össze. A receptek saját belső hozzávalólistával rendelkeznek, amelyben minden elem egy alapanyagból és mennyiségből áll. A felhasználó később kereshet, törölhet, listázhat, illetve mentheti vagy betöltheti az adatokat. A teljes rendszer lényege az objektumorientált modellezés: minden valós fogalomhoz külön osztály tartozik, az osztályok között pedig logikus kapcsolat van. Ezáltal a program átlátható, bővíthető és könnyen tesztelhető marad.

Recipeingredient osztály

getAmount()

Ez a metódus visszaadja a hozzávaló mennyiségét, amelyet a konstruktor során állítottunk be. A visszatérési érték double, mert a mennyiség lehet tört szám is (például 2.5 dl). A metódus nem módosítja az objektum állapotát, csak lekérdezi az adatot. Ez a getter metódus a kapszulázás elvét követi, mivel a mező közvetlenül nem érhető el kívülről.

getIngredient()

Ez a metódus visszaadja azt az Ingredient objektumot, amely a hozzávalóhoz tartozik. Ez lehetővé teszi, hogy a program más részei hozzáférjenek az alapanyag teljes adataihoz (például név, mértékegység). Így nem kell külön tárolni minden adatot, hanem az objektumhivatkozáson keresztül érhető el minden szükséges információ. Ez az objektumorientált programozás egyik alapelve: objektumok egymásra hivatkoznak.

getName()

Ez a metódus az alapanyag nevét adja vissza, de nem közvetlen mezőből, hanem az Ingredient objektumból. Gyakorlatilag egy „delegáló” metódus, amely továbbhívja az ingredient.getName() metódust. Ez kényelmesebbé teszi a használatot, mert nem kell mindig külön lekérni az Ingredient objektumot. A TreeMap kulcsaként is ez a név kerül felhasználásra más osztályokban.

toString()

Ez a metódus szöveges formában adja vissza a hozzávalót, például: rum - 2.0 l. A metódus az alapanyag nevét, a mennyiséget és a mértékegységet egyetlen olvasható stringgé alakítja. Ez különösen hasznos listázáskor, amikor a felhasználónak szeretnénk megjeleníteni az adatokat. A toString() felüldefiniálása biztosítja, hogy az objektum kiírásakor értelmes, ember által olvasható formát kapjunk.

RecipeDatabase osztály

findIngredient()

Ez a metódus az alapanyagok között keres név alapján. A TreeMap kulcsa az alapanyag neve, ezért a get() metódussal közvetlenül lekérhető a hozzá tartozó objektum. Ha létezik ilyen nevű alapanyag, akkor visszaadja az Ingredient objektumot, ellenkező esetben null értéket ad vissza. Ez a gyors keresés a kulcs-érték alapú tárolás egyik fő előnye.

addIngredient()

Ez a metódus egy új alapanyagot ad hozzá az adatbázishoz. Az alapanyag neve kulcsként kerül eltárolásra, így később könnyen visszakereshető. Ha már létezik ugyanilyen nevű alapanyag, akkor az új felülírja a régit. Ez a működés a TreeMap sajátossága.

addRecipe()

Ez a metódus egy új receptet ment el az adatbázisban. A recept neve lesz a kulcs, így a felhasználó név alapján tudja majd visszakeresni. Hasonlóan az alapanyagokhoz, itt is felülíródik az adat, ha ugyanazzal a névvel már létezik recept. Ez egyszerű, de hatékony adatkezelést biztosít.

deleteRecipe()

Ez a metódus törli a megadott nevű receptet az adatbázisból. A TreeMap remove() metódusát használja, amely a kulcs alapján távolítja el az elemet. Ha a recept nem létezik, akkor nem történik semmi. Ez biztonságos működést biztosít, mert nem dob hibát.

findRecipe()

Ez a metódus egy receptet keres név alapján. Ha a recept létezik, visszaadja a Recipe objektumot, így azon további műveletek végezhetők. Ha nem található, akkor null értékkel tér vissza. Ez lehetővé teszi, hogy a hívó kód eldöntse, hogyan kezelje a hiányzó adatot.

listIngredientsOfRecipe()

Ez a metódus egy adott recept hozzávalóit listázza ki. Először megkeresi a receptet az adatbázisban, majd ha létezik, meghívja annak listIngredients() metódusát. Ha a recept nem található, akkor hibaüzenetet ír ki a felhasználónak. Ez a metódus összekapcsolja az adatbázis és a recept működését.

listRecipesByName()

Ez a metódus az adatbázisban tárolt recepteket név szerinti sorrendben listázza ki. A rendezés automatikusan történik, mivel a receptek TreeMap adatszerkezetben vannak tárolva, amely a kulcsok alapján rendezi az elemeket. A metódus végigiterál az összes recepten, és kiírja azok nevét.

listRecipesByIngredientCount()

Ez a metódus a recepteket a bennük található hozzávalók száma alapján rendezi és listázza. A recepteket egy listába másolja, majd egy rendezési algoritmus segítségével növekvő sorrendbe állítja őket a hozzávalók száma szerint. A metódus minden recept mellett megjeleníti a hozzávalók számát is. Ez segíti a receptek összehasonlítását összetettség alapján.

saveToFile()

Ez a metódus az egész adatbázist fájlba menti szerializáció segítségével. Az ObjectOutputStream segítségével a teljes objektum (beleértve az összes receptet és alapanyagot) egyben kerül mentésre. Ez egyszerűbb és biztonságosabb, mint kézzel fájlba írni az adatokat. A metódus hibát dobhat (IOException), ha probléma történik az írás során.

loadFromFile()

Ez a metódus visszatölti az adatbázist egy fájlból. Az ObjectInputStream segítségével beolvassa a korábban elmentett objektumot, majd visszaalakítja RecipeDatabase típusúvá. A metódus statikus, ezért példány nélkül is meghívható. Hibát dobhat, ha a fájl nem létezik vagy az adat nem megfelelő formátumú.

Recipe osztálya

getName()

Ez a metódus visszaadja a recept nevét. Nem módosítja az objektum állapotát, csak lekérdezi az adatot. A metódus fontos szerepet játszik például a RecipeDatabase-ben, ahol a recept neve kulcsként kerül felhasználásra. Ez biztosítja a receptek egyedi azonosítását.

getIngredients()

Ez a metódus visszaadja a recepthez tartozó összes hozzávalót tartalmazó TreeMap-et. Így a program más részei közvetlenül hozzáférhetnek a teljes hozzávalólistához. A visszatérési érték egy összetett adatszerkezet, amely kulcs-érték párokban tárolja az adatokat. Ez a metódus főként akkor hasznos, ha további feldolgozás szükséges.

addIngredient()

Ez a metódus egy új hozzávalót ad hozzá a recepthez. A TreeMap kulcsa a hozzávaló neve (h.getName()), így biztosított a gyors keresés. Ha már létezik ilyen nevű hozzávaló, akkor az új érték felülírja a régit. Ez egyszerű és hatékony adatkezelést tesz lehetővé.

removeIngredient()

Ez a metódus törli a megadott nevű hozzávalót a receptből. A TreeMap remove() metódusát használja, amely a kulcs alapján távolítja el az elemet. Ha a hozzávaló nem létezik, a művelet nem okoz hibát. Ez biztonságos működést eredményez.

findIngredient()

Ez a metódus egy adott nevű hozzávalót keres a recepten belül. Ha a hozzávaló megtalálható, akkor visszaadja a hozzá tartozó RecipeIngredient objektumot. Ha nem létezik, akkor null értékkel tér vissza. Ez lehetővé teszi a további ellenőrzést vagy feldolgozást.

listIngredients()

Ez a metódus kiírja a recept összes hozzávalóját a konzolra. A TreeMap értékein végigiterál, és minden elemet a toString() metódus segítségével jelenít meg. Ez felhasználóbarát módon mutatja meg a hozzávalókat. A metódus főként a felhasználói felületen történő megjelenítéshez szükséges.

toString()

Ez a metódus a teljes receptet egy szöveges formában adja vissza. Tartalmazza a recept nevét és az összes hozzávalót. A `ingredients.values()` segítségével az összes hozzávaló megjelenik a kimenetben. Ez hasznos például debugoláskor vagy gyors kiírásnál.

Menu osztály

isNumber(String s)

Ez a metódus azt ellenőrzi, hogy a paraméterként kapott szöveg átalakítható-e egész számmá. A try-catch blokkban az `Integer.parseInt()` segítségével megpróbálja számmá alakítani a szöveget. Ha ez sikerül, akkor `true` értékkel tér vissza, ha pedig hibát kap, akkor `false` értéket ad vissza. Ez a metódus főként a felhasználói bemenetek ellenőrzésére szolgál.

menu_print()

Ez a metódus a program főmenüjének szövegét adja vissza. A menüben szerepelnek a választható funkciók, például új alapanyag felvétele, új recept létrehozása, recept törlése, listázás, mentés, betöltés és kilépés. A metódus nem ír ki közvetlenül a konzolra, hanem egy `String` értéket ad vissza. Ezt a `Main` osztály tudja kiírni a felhasználónak.

userInput()

Ez a metódus a felhasználó menüválasztását kéri be. A bemenetet először szöveggént olvassa be, majd az `isNumber()` metódussal ellenőrzi, hogy valóban szám-e. Ha a bemenet szám, akkor egész számmá alakítja és visszaadja. Ha nem számot adott meg a felhasználó, akkor hibaüzenetet ír ki, és újra bekéri az értéket.

InputIngridient()

Ez a metódus egy új alapanyag adatait kéri be a felhasználótól. A felhasználónak két adatot kell megadnia szóközzel elválasztva: az alapanyag nevét és a mértékegységét, például `rum l`. A metódus ellenőrzi, hogy pontosan két adat érkezett-e, egyik sem üres, és egyik sem szám. Ha a bevétel helyes, akkor egy `ArrayList<String>` listában visszaadja a két értéket.

InputRecipe()

Ez a metódus egy recepthez tartozó hozzávaló adatait kéri be. A felhasználónak egy alapanyag-nevet és egy mennyiséget kell megadnia, például `rum 2`. A metódus ellenőrzi, hogy az első adat szöveg

legyen, a második pedig szám. Ha a bemenet megfelelő, akkor a két adatot egy `ArrayList<String>` listában adja vissza.

newRecipe()

Ez a módszer egy recept nevét kéri be a felhasználótól. A módszer addig ismétli a bekérést, amíg a felhasználó nem üres szöveget ad meg. Ez azért fontos, mert üres nevű receptet nem érdemes eltárolni az adatbázisban. Ha a felhasználó érvényes nevet ír be, a módszer visszatér ezzel a névvel.

newinput()

Ez egy egyszerű segédmetódus, amely egy sort olvas be a konzolról. Főként igen/nem típusú kérdéseknél használható, például amikor a program megkérdezi, hogy a felhasználó szeretne-e még alapanyagot vagy hozzávalót hozzáadni. A módszer nem végez külön ellenőrzést, csak visszaadja a beírt szöveget. Ez egyszerűbbé teszi az ismétlődő inputkezelést.

Ingridient osztálya

getName()

Ez a módszer visszaadja az alapanyag nevét. Nem módosítja az objektum állapotát, kizárólag lekérdezi az adatot. A név fontos szerepet játszik, mivel más osztályokban (például `TreeMap` kulcsként) azonosításra használják. Ez a módszer a kapszulázás elvét követi.

getUnit()

Ez a módszer visszaadja az alapanyag mértékegységét. A visszatérési érték egy `String`, például „l”, „g” vagy „db”. Ez az információ szükséges ahhoz, hogy a hozzávalók helyesen jelenjenek meg a felhasználó számára. A módszer csak lekérdezi az adatot, nem módosítja azt.

toString()

Ez a módszer az alapanyagot szöveges formában adja vissza. A visszatérési érték tartalmazza az alapanyag nevét és a mértékegységet, például: rum (l). Ez megkönnyíti az objektumok kiírását és megjelenítését. A módszer felüldefiniálja az alapértelmezett `toString()` működést, hogy ember által olvasható formát biztosítson.

Appservice osztály

NewIngredient()

Ez a metódus az új alapanyagok felvitelét kezeli. A felhasználótól a Menu osztály segítségével bekéri az alapanyag nevét és mértékegységét, majd ezekből létrehoz egy új Ingredient objektumot. Az elkészült alapanyagot eltárolja a RecipeDatabase adatbázisban. A metódus ciklusban működik, így a felhasználó egymás után több alapanyagot is hozzáadhat.

NewRecipe()

Ez a metódus új recept létrehozására szolgál. Először bekéri a recept nevét, majd létrehoz egy új Recipe objektumot. Ezután a felhasználó megadhatja a recept hozzávalóit és azok mennyiségét. A metódus ellenőrzi, hogy a megadott alapanyag létezik-e az adatbázisban, és csak létező alapanyagból hoz létre recept-hozzávalót.

DeleteRecipe()

Ez a metódus egy meglévő recept törlését végzi. A felhasználótól bekéri a törlendő recept nevét, majd megkeresi azt az adatbázisban. Ha a recept létezik, akkor törli a RecipeDatabase-ből. Ha nem található ilyen recept, akkor hibaüzenetet ír ki a felhasználónak.

ListIngredients()

Ez a metódus egy kiválasztott recept hozzávalóit listázza ki. A felhasználó megadja a recept nevét, majd a program megkeresi azt az adatbázisban. Ha a recept létezik, akkor kiírja a recepthez tartozó összes hozzávalót. Ha a recept nem található, akkor a program figyelmeztető üzenetet jelenít meg.

SaveToFile()

Ez a metódus az adatbázis fájlba mentését indítja el. Meghívja a RecipeDatabase osztály saveToFile() metódusát, amely szerializációval elmenti az aktuális adatokat. A mentés az adatbázis.dat fájlba történik. A metódus visszajelzést ad a felhasználónak a mentés megkezdéséről és sikerességéről.

LoadFromFile()

Ez a metódus az adatbázis fájlból történő betöltését végzi. Meghívja a RecipeDatabase osztály loadFromFile() metódusát, amely visszaolvassa a korábban elmentett adatokat. A metódus egy új RecipeDatabase objektummal tér vissza, amely tartalmazza a betöltött recepteket és alapanyagokat. Ez biztosítja, hogy a program újraindítás után is képes legyen a korábbi adatokkal dolgozni.

Felhasználói kézikönyv – Használati útmutató

A program egy konzolos felületen működő italrecept-kezelő alkalmazás, amely lehetővé teszi alapanyagok és receptek egyszerű nyilvántartását. A felhasználó egy menürendszeren keresztül tudja kezelni az adatokat, ahol minden művelet egy számmal választható ki. A program működése ciklikus, ha belépünk egy menü alrészben folyamatosan kérdezi a program minden egyes művelet után hogy bent maradjunk-e ellenkező esetben minden művelet után visszatér a főmenübe, amíg a felhasználó ki nem lép.

A program indítása

A program a Main osztályból indul, ahol létrejön egy üres adatbázis, valamint a menükezelő objektum. Ezután egy folyamatos ciklusban megjelenik a menü, és a felhasználó kiválaszthatja a kívánt műveletet. A választás egy szám megadásával történik, amelyet a program ellenőriz, így hibás bemenet esetén újra bekéri az értéket.

Főmenü működése

A program az alábbi menüpontokat kínálja:

1. Új alapanyag hozzáadása
2. Új recept hozzáadása
3. Recept törlése
4. Hozzávalók listázása
5. Receptek listázása név szerint
6. Receptek listázása hozzávalók száma szerint
7. Mentés fájlba
8. Betöltés fájlból
9. Kilépés

A felhasználó a megfelelő szám beírásával tud választani. A program minden esetben ellenőrzi, hogy a bemenet szám-e, így elkerülhetők a hibás működések.

Új alapanyag hozzáadása

Írd be az **1-es számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó új alapanyagot rögzíthet az adatbázisban. A program két adatot kér be: az **alapanyag** nevét és a **mértékegységét, szóközzel elválasztva**, például: **rum l**. A bevitel ellenőrzésén meggy keresztül, így csak helyes formátum esetén kerül mentésre. A **felhasználó több alapanyagot is hozzáadhat egymás után, ehhez i vagy igen szöveget kell beírni**.

Új recept hozzáadása

Írd be a **2-es számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó új receptet hozhat létre. Először meg kell adni a recept nevét, például: **Mojito**. Ezután a program bekéri a hozzávalókat, ahol az alapanyag nevét és a mennyiséget kell megadni szóközzel elválasztva, például: **rum 2**. A program **csak olyan alapanyagot fogad el, amely már szerepel az adatbázisban!!!** Több hozzávaló is megadható, amelyhez **i vagy igen** beírásával lehet folytatni.

Recept törlése

Írd be a **3-as számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó törölhet egy meglévő receptet. A program **bekéri a recept nevét**, majd megkeresi azt az adatbázisban. Ha a recept létezik, akkor törlésre kerül. Ha nem található ilyen nevű recept, a program hibaüzenetet jelenít meg.

Hozzávalók listázása

Írd be a **4-es számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó megtekintheti egy adott recept hozzávalóit. A program **bekéri a recept nevét**, majd kilistázza az összes hozzávalót mennyiséggel és mértékegységgel együtt. Ha a megadott recept nem létezik, a program figyelmeztetést ad.

Receptek listázása név szerint

Írd be a **5-as számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó megtekintheti az adatbázisban tárolt recepteket név szerinti sorrendben. A program a recepteket ABC sorrendben jeleníti meg, mivel a receptek tárolása TreeMap segítségével történik. Ez átláthatóbbá teszi a receptek keresését és ellenőrzését.

Receptek listázása hozzávalók száma szerint

Írd be a **6-es számot**, majd nyomj **Entert**. Ebben a menüpontban a felhasználó a recepteket a hozzájuk tartozó hozzávalók száma alapján listázhatja ki. A program minden recept mellett megjeleníti, hogy hány hozzávalót tartalmaz. Ez segít összehasonlítani az egyszerűbb és összetettebb recepteket.

Mentés fájlba

Írd be az 7-ös számot, majd nyomj Entert. A program ekkor elmenti az aktuális adatbázist egy fájlba. A mentés automatikusan történik az **adatbazis.dat** nevű fájlba. A művelet után a program visszajelzést ad a sikeres mentésről.

Betöltés fájlból

Írd be a 8-os számot, majd nyomj Entert. Ebben a menüpontban a program betölti a korábban elmentett adatokat az **adatbazis.dat** fájlból. A betöltés után az adatbázis tartalma visszaáll az utolsó mentett állapotra. Ha a fájl nem létezik vagy hiba történik, a program hibaüzenetet ír ki.

Kilépés

Írd be a 9-es számot, majd nyomj Entert. A program ekkor leáll és kilép a futásból. Kilépés előtt érdemes menteni az adatokat, hogy ne vesszenek el.

Kép Forrás: [link](#)